COMPUTER VISION CSL7360 | ASSIGNMENT 2

Cost-Efficient Segmentation under a Compute Budget

A Compact Fully Convolutional Network with
Validation-Guided Inference and Storage Optimization

Aashcharya Gorakh B23ES1001

IIT Jodhpur

1 November 2026

Abstract

This report presents a binary salient-object segmentation pipeline designed for an average single-core CPU budget of 50 ms per image. A compact fully convolutional network is trained from scratch on DUTS-TR, with checkpoint and deployment settings selected using a separate validation split. The selected system combines an encoder–decoder architecture, skip connections, global context, TorchScript execution optimization, and uncompressed mixed-precision parameter storage.

On all 5,019 DUTS-TE images, the final model achieves mIoU = 0.7083, BF1 = 0.4726, and $Q = 0.6376$. Average CPU inference takes 10.821 ms, and the parameter file occupies 499,643 bytes. The resulting CES = 0.6024 improves on the recorded baseline of 0.5920 by approximately 1.76% relative. Six independently annotated college photographs provide an additional test-only assessment.

1 Introduction and Objective

Salient-object segmentation identifies the main object or objects in an image and separates them from the background at pixel level. Under a compute budget, segmentation quality must be balanced against execution time and deployment size.

The assignment evaluates this balance through the Cost-Efficient Segmentation score. The implementation follows **Track B**: a small fully convolutional network trained without externally pretrained weights.

The architecture uses fully convolutional prediction and encoder–decoder skip connections [2, 3]. Depthwise and pointwise refinement provide a lightweight convolutional component [4].

1.1 Engineering goals

The pipeline is designed to:

1. produce a 256×256 binary mask for every photograph;
2. remain below 50 ms average single-thread CPU inference time;
3. retain useful region overlap and boundary accuracy;
4. store learned inference values in one parameter file;
5. support reproducible training, evaluation, and prediction.

The experiments evaluate input resolution, decision thresholds, flip averaging, execution optimization, storage precision, and network width using validation measurements.

Table 1. Final DUTS-TE deployment summary.

CES	CPU time	Parameter file	Parameters
0.6024	10.821 ms/image	499,643 bytes	466,274

2 Dataset and Experimental Protocol

2.1 DUTS and the fixed split

DUTS contains 10,553 training images and 5,019 test images with salient-object annotations [1]. DUTS-TR is divided using sorted filenames and a seed-42 permutation.

Table 2. Dataset partition and experimental role.

Subset	Images	Role
DUTS-TR training	9,497	Gradient-based learning and augmentation
DUTS-TR validation	1,056	Checkpoint and deployment-setting selection
DUTS-TE	5,019	Frozen-model benchmark evaluation
College photographs	6	Additional test-only generalisation evaluation

The training count is $\lfloor 0.9 \times 10{,}553 \rfloor = 9{,}497$. Exact filenames and a dataset fingerprint are retained in `data/split.json`. Images and masks are matched by basename across separate directories. Missing annotations, unexpected counts, and changed fingerprints produce explicit errors.

DUTS-TE and the college photographs are excluded from gradient training, checkpoint selection, threshold selection, and early stopping. The original baseline had already been reported on DUTS-TE before the improvement pass. The final result is therefore a repeated frozen-model evaluation, with subsequent candidate decisions based on validation measurements.

2.2 Preprocessing

Every source photograph is resized to 256×256 using `cv2.INTER_AREA`. Annotations use `cv2.INTER_NEAREST`. Non-square photographs are intentionally squashed to the required square resolution.

The binary annotation is defined by

$$y(u) = \begin{cases} 1, & m(u) \geq 128, \\ 0, & m(u) < 128. \end{cases}$$

OpenCV BGR images are converted to RGB tensors and normalized to $[0, 1]$. The network then internally resizes its tensor to 160×160; this additional resize is included in timing.

A lossless uint8 training/validation cache reduces repeated image decoding. Cached and uncached sample tensors were compared and matched exactly. The timed predictor does not access this cache.

2.3 Augmentation

Training applies a synchronized horizontal flip to the image and mask with probability 0.5. Photometric changes affect the image only:

$$I' = \mathrm{clip}(\alpha I + \beta, 0, 255), \qquad \alpha \sim \mathcal{U}(0.8, 1.2), \quad \beta \sim \mathcal{U}(-12, 12).$$

Validation and test images are not randomly augmented.

2.4 College photographs and provenance

The photographer confirmed collecting the six photographs around the college in daylight and at night; the stall was inside the college. The received WhatsApp JPEG files are preserved unchanged. Source filenames, renamed filenames, target definitions, and SHA-256 hashes are recorded in `captured/manifest.json`.

The set covers relative near/far object scale, cluttered backgrounds, and partial occlusion of the stall chair by merchandise. Image-edge truncation is recorded separately from occlusion. Exact per-image tube lighting and outdoor capture are not asserted.

Reference masks were independently traced on the resized photographs before inspecting predictions. Fine spokes, slats, and furry edges remain approximate. All six photographs are retained in evaluation; none is removed to increase the reported score.

3 Network Architecture

3.1 Overview

The selected FCN contains **466,274 trainable parameters**. Five encoder stages reduce the internal 160×160 image to a 5×5 representation. Four decoder stages recover spatial information using concatenated encoder features.

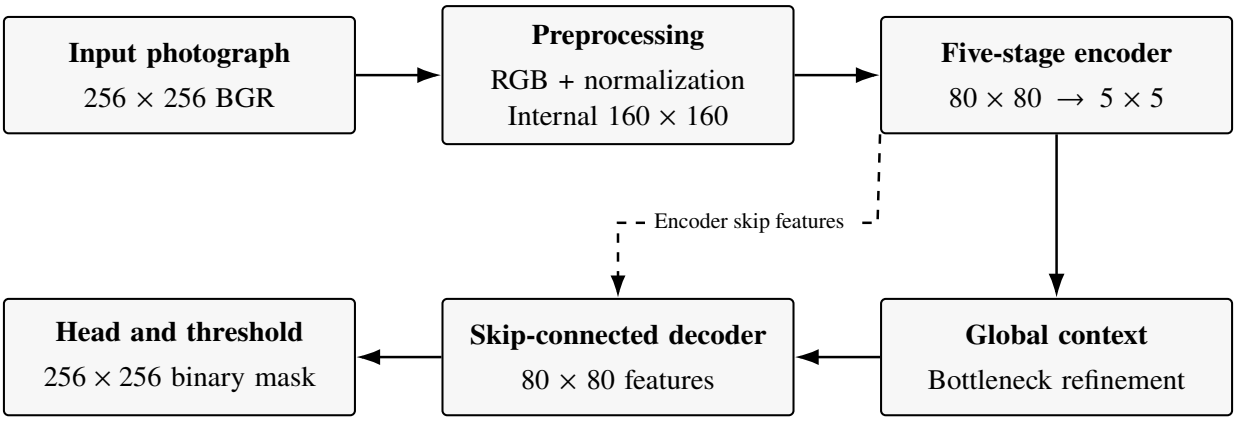


Figure 1. Timed prediction pipeline. Solid arrows show the main forward path. The dashed route represents encoder features supplied to the decoder. Parameter loading and TorchScript preparation precede timing.

Table 3. Selected full-width architecture.

Stage	Resolution	Channels	Operation
Assignment input	256×256	3	Photograph in memory
Internal input	160×160	3	Bilinear resize
Encoder 1	80×80	16	Stride-two block
Encoder 2	40×40	32	Stride-two block
Encoder 3	20×20	64	Stride-two block
Encoder 4	10×10	96	Stride-two block
Encoder 5	5×5	128	Stride-two block
Context	5×5	128	Broadcast pooled context
Decoder 4	10×10	64	Upsampling and skip
Decoder 3	20×20	48	Upsampling and skip
Decoder 2	40×40	32	Upsampling and skip
Decoder 1	80×80	16	Upsampling and skip
Class head	80×80	2	1×1 convolution
Output	256×256	2	Upsampled logits

3.2 Blocks, context, and prediction

Each block contains a standard 3×3 convolution followed by depthwise 3×3 and pointwise 1×1 refinement. Batch normalization and ReLU follow each convolution.

The context branch adds a projected pooled vector:

$$\widetilde{e}_5 = e_5 + \mathrm{Conv}_{1\times1}(\mathrm{GAP}(e_5)) .$$

Decoder stages concatenate upsampled features and encoder skips:

$$d_\ell = B_\ell \left([\mathrm{Up}(d_{\ell+1}), e_\ell]\right) .$$

The binary output is

$$\widehat{y}(u) = \mathbf{1}[z_1(u) - z_0(u) > 0].$$

Ties are background. No morphology or additional segmentation postprocessing is applied.

4 Training Procedure

4.1 Objective

The full-width network is trained from random initialization using pixelwise cross entropy and foreground Soft Dice:

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \mathcal{L}_{\mathrm{Dice}}.$$

For a batch of B images,

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{BHW} \sum_{n=1}^{B} \sum_{u} \log p_{n,u,y_{n,u}},$$

and

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{1}{B} \sum_{n=1}^{B} \frac{2 \sum_u p_{n,u,1} y_{n,u} + 1}{\sum_u p_{n,u,1} + \sum_u y_{n,u} + 1}.$$

Cross entropy supplies pixelwise supervision, while Soft Dice encourages foreground overlap.

Table 4. Training configuration of the selected network.

Setting	Value
Initialization	Random; no external pretrained weights
Optimizer	AdamW [**?**]
Initial learning rate	2×10^{-3}
Weight decay	10^{-4}
Schedule	Cosine learning-rate decay
Batch size	32
Completed epochs	40
Selected checkpoint	Epoch 31, highest validation Q
Early-stopping patience	10 non-improving epochs
Training device	NVIDIA GeForce RTX 3050 Laptop GPU
Training precision	CUDA mixed precision

4.2 Checkpoint selection

After each epoch, the fixed validation set determines

$$Q_{\mathrm{val}} = 0.70\,\mathrm{mIoU}_{\mathrm{val}} + 0.30\,\mathrm{BF1}_{\mathrm{val}}.$$

The best validation checkpoint is retained rather than automatically using the final epoch. Seeds are fixed, although GPU numerical results can vary across hardware and software builds.

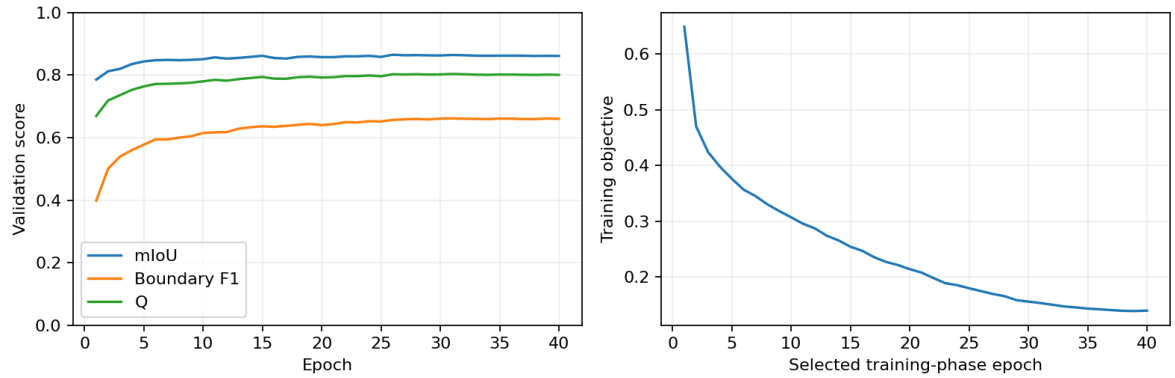


Figure 2. Training objective and validation history. The left panel reports mIoU, BF1, and Q; the right panel shows the training objective. Validation quality largely plateaus while training loss continues to decline.

The saved checkpoint is reloaded for final CPU evaluation. Consequently, deployment measurements reflect the exported parameter representation rather than only the in-memory training model.

5 Inference and Storage Optimization

5.1 Validation-only inference search

A fixed grid evaluates internal resolutions $\{128, 160, 192, 224, 256\}$, flip averaging on/off, and logit biases $\{-0.75, -0.5, -0.25, 0, 0.25, 0.5, 0.75\}$. Quality uses all 1,056 validation images. Initial CPU estimates repeat a validation photograph 64 times after warm-up.

The retained full-width configuration uses 160×160 input, no flip averaging, and zero logit bias.

5.2 Execution and parameter storage

Allowed TorchScript tracing, freezing, and inference optimization reduce execution overhead. Convolution kernels use per-output-channel storage quantization:

$$s_o = \frac{\max_j |W_{o,j}|}{127}, \qquad q_{o,j} = \text{clip}\left(\text{round}(W_{o,j}/s_o), -127, 127\right).$$

Reconstruction uses $\widehat{W}_{o,j} = s_o q_{o,j}$. A zero-valued channel uses unit scale and zero quantized values.

Quantized values are int8; scales are float32. Biases and floating-point normalization state retain float16 storage. Weights are reconstructed into a float32 CPU model before timing. This is storage quantization, not int8 convolution execution.

Same-dtype tensors are concatenated into raw arrays to reduce archive headers. Tensor payloads are not compressed. All weights, scales, architecture metadata, and inference settings remain in the single parameter file.

5.3 Architecture comparison

A half-width candidate contains 118,578 parameters and is trained from random initialization for 40 epochs. A three-quarter-width candidate contains 263,786 parameters and is pruned from this project's own scratch-trained model, followed by ten fine-tuning epochs.

The experimental candidates additionally use a soft-target cross entropy from the frozen within-project teacher:

$$\mathcal{L}_{\text{candidate}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{Dice}} + 0.5\mathcal{L}_{\text{soft}}.$$

No external pretrained weights or additional images are introduced.

Table 5. Paired full-validation comparison. INT8-HC retains float16 head/context kernels. Every candidate processes every validation image in rotating order.

Candidate	Q	CPU ms	Bytes	CES
Full / INT8	**0.803310**	**11.290**	**499,693**	**0.757463**
Half / FP16	0.768674	6.880	255,997	0.742828
Half / INT8	0.768391	7.013	144,297	0.744191
Half / INT8-HC	0.768404	6.910	148,213	0.744450
Half / INT8 + flip	0.781995	13.805	144,287	0.736123
Three-quarter / FP16	0.778247	8.671	548,029	0.741087
Three-quarter / INT8	0.778534	9.233	293,561	0.744333
Three-quarter / INT8-HC	0.778568	9.337	302,533	0.743876
Three-quarter / INT8 + flip	0.790572	18.259	293,551	0.727300

The full-width model provides the highest validation CES. Narrower models are faster, but their quality loss offsets their cost reduction. Flip averaging improves Q without improving CES.

The final exported file occupies 499,643 bytes. Its small difference from the candidate-file size is due to serialization naming overhead. The retained filename `params/model_fp16.pt` is a legacy name; its contents are mixed precision.

6 Evaluation Metrics and CPU Timing

6.1 Global mean intersection over union

One confusion matrix is accumulated over all images and pixels:

$$C_{ij} = \sum_{n,u} \mathbf{1}[y_n(u) = i \wedge \widehat{y}_n(u) = j].$$

Rows are reference classes and columns are predictions. For class c,

$$\mathrm{IoU}_c = \frac{C_{cc}}{\sum_j C_{cj} + \sum_i C_{ic} - C_{cc}}, \qquad \mathrm{mIoU} = \frac{\mathrm{IoU}_0 + \mathrm{IoU}_1}{2}.$$

Set-level mIoU is therefore not the mean of per-image IoUs. A zero-union class receives zero; both classes occur in the DUTS aggregate evaluations.

6.2 Boundary F1

A boundary is a foreground pixel with a background neighbour directly above, below, left, or right. Pixels outside the image count as background.

A predicted boundary pixel matches when a reference boundary pixel lies in its 5×5 window. Recall applies the reverse matching direction. For nonempty boundaries,

$$F_1 = \begin{cases} \dfrac{2PR}{P+R}, & P + R > 0, \\ 0, & P + R = 0. \end{cases}$$

If both boundaries are empty, F1 is one; if exactly one is empty, it is zero. BF1 is the mean image-level F1. Boundary extraction and neighbourhood matching use direct NumPy operations, without library metric or morphology routines.

6.3 Quality and CES

$$Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1}.$$

$$\mathrm{CES} = \begin{cases} Q\left[1 - 0.20\dfrac{T_{\mathrm{CPU}}}{50} - 0.10\min\left(1, \dfrac{S_{\mathrm{disk}}}{4{,}194{,}304}\right)\right], & T_{\mathrm{CPU}} \le 50\,\mathrm{ms}, \\ 0, & \text{otherwise}. \end{cases}$$

6.4 Timing scope

Timing begins with a 256×256 uint8 BGR photograph in memory and ends with a 256×256 uint8 mask containing zero and one.

Table 6. Operations inside and outside the timed interval.

Included	Excluded
Colour conversion and normalization	File reading and writing
Tensor creation and internal resizing	Initial source-photo resize to 256×256
Network execution and output upsampling	Parameter loading and reconstruction
Thresholding and output-array conversion	TorchScript preparation and training

After one real-photo warm-up, `time.process_time()` measures each prediction separately. DUTS-TE time is the sum divided by 5,019. Metrics are computed outside the timed interval.

Before numerical imports, `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `VECLIB_MAXIMUM_THREADS` are set to one. OpenCV and PyTorch intra-op/inter-op limits are also one. Timed inference creates no subprocesses or worker processes.

Table 7. Execution environment.

CPU	AMD Ryzen 7 4800H with Radeon Graphics
Training GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GiB
Software	Python 3.12.1; PyTorch 2.7.1+cu118
Libraries	NumPy 1.26.4; OpenCV 4.11.0

7 Quantitative Results

Table 8. Final measurements using the same frozen checkpoint. Only DUTS-TE CES is the common assignment benchmark; other CES values are diagnostic.

Set	Images	mIoU	BF1	Q	CPU ms	Bytes	CES
Validation	1,056	0.8641	0.6614	0.8033	12.533	499,643	0.7535
DUTS-TE	**5,019**	**0.7083**	**0.4726**	**0.6376**	**10.821**	**499,643**	**0.6024**
College photos	6	0.7639	0.4507	0.6699	7.812	499,643	0.6410

7.1 Baseline comparison

Table 9. Observed DUTS-TE baseline-to-final comparison.

Quantity	Baseline	Final
mIoU	0.707915	0.708287
BF1	0.472321	0.472643
Q	0.637237	0.637594
CPU ms/image	11.883	10.821
Parameter bytes	985,783	499,643
CES	0.591971	0.602400

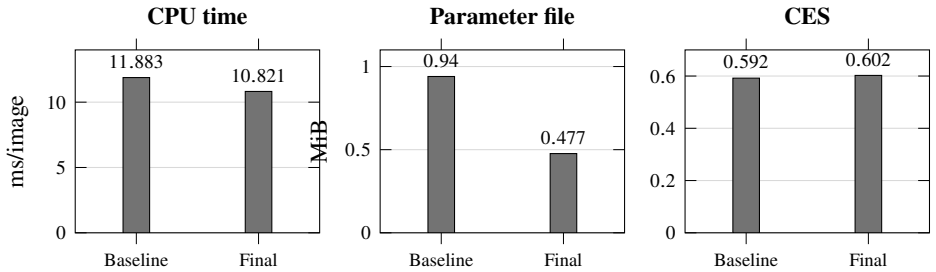


Figure 3. Measured deployment costs and CES. Axes begin at zero. Baseline and final timings belong to separate execution windows.

The observed CES gain is **1.76% relative**, with a **49.32% reduction** in parameter-file size. The final CPU mean uses approximately 21.6% of the permitted budget:

$$\text{CES} \approx 0.637594(1 - 0.04329 - 0.01191) = 0.60240.$$

7.2 Confusion-matrix interpretation

The final DUTS-TE confusion matrix is

$$C = \begin{bmatrix} 261\,995\,359 & 18\,211\,965 \\ 13\,591\,526 & 35\,126\,334 \end{bmatrix}.$$

It contains 328,925,184 evaluated pixels. Background IoU is 0.89175 and object IoU is 0.52482, giving mIoU 0.70829.

The validation-to-test quality gap remains. Timing depends on background activity and power/thermal conditions. The six-image captured-set latency is less stable than the complete DUTS-TE average.

8 Captured-Photo Evaluation: Part I

All six photographs contribute to the reported evaluation. The display order highlights strong examples first without changing the evaluated set. Each triptych shows **input**, **reference-mask overlay**, and **predicted-mask overlay**, from left to right.

Reference overlays are green and predictions are cyan. Photographs and overlays retain their original colours.

Table 10. Individual captured-photo measurements. Set-level mIoU uses pooled confusion counts rather than the mean of these per-image mIoUs.

Photograph	mIoU	BF1	Q
Body-wash bottle	0.9652	0.8793	0.9394
Turtle lamp	0.8553	0.5991	0.7785
Campus cat	0.7582	0.4207	0.6569
Giraffe toy	0.7831	0.3505	0.6533
Bicycle	0.4775	0.2642	0.4135
Stall chair	0.5038	0.1907	0.4099



Figure 4. Body-wash bottle. The reference includes the bottle and cap, excluding the hand. Input, reference overlay, and prediction overlay are shown left to right.



Figure 5. Turtle lamp. The reference includes the complete lamp and shade, excluding cast shadow. Input, reference overlay, and prediction overlay are shown left to right.



Figure 6. Campus cat. The reference includes the visible cat, excluding background and cast shadow. Input, reference overlay, and prediction overlay are shown left to right.

9 Captured-Photo Evaluation: Part II

The remaining photographs provide variation in object structure, scale, texture, and background complexity. Foreground definitions are fixed before predictions are inspected.

Table 11. Foreground definitions and annotation scope.

Photograph	Included target	Excluded content
Giraffe toy	Soft toy and visible appendages	Paper tag, hand, and background
Bicycle	Visible frame, tyres, and spokes	Open wheel/frame spaces and background
Stall chair	Central plastic chair	Merchandise and other chairs



Figure 7. Giraffe toy. Input, independently prepared reference overlay, and predicted overlay are shown left to right.



Figure 8. Bicycle. Background holes are retained in the reference; tyres and spokes use manually positioned geometric approximations.



Figure 9. Stall chair. The distant central chair is the specified target. Foreground merchandise partially occludes its lower structure.

Received source files are preserved without re-encoding. Matching basenames are used across `captured/original/`, `captured/images_256/`, and `captured/masks_256/`. Fine annotation boundaries remain approximate, particularly around spokes, slats, and fur.

10 Discussion and Reproducibility

10.1 Qualitative interpretation

The body-wash bottle is the strongest captured example: the main silhouette is recovered accurately, with small hand/finger false positives. The lamp also demonstrates useful localization of a composite object shape.

Two challenging cases clarify the remaining limitations. For the **bicycle**, predicted regions fill open wheel/frame space rather than preserving thin topology. Internal downsampling and smooth upsampling favour broad regions, while approximate spoke annotations add boundary uncertainty.

For the **stall chair**, the small target competes with nearby chairs and merchandise. The prediction selects distracting regions and misses portions of the specified foreground. Object scale, clutter, and partial occlusion make the saliency decision ambiguous.

These cases remain in the aggregate evaluation. Neither reference masks nor image selection are altered to match predictions.

10.2 Reproducibility and verification

The submission retains:

- the fixed train/validation split and dataset fingerprint;
- training histories and validation-selection records;
- set-level and per-image metrics, confusion matrices, and times;
- checkpoint identity and unchanged source-image hashes;
- independent coordinate annotations and their rasterizer.

Tests cover the boundary oracle, empty/full/border cases, pooled mIoU, CES, legacy and packed checkpoints, architecture reconstruction, and the binary-mask contract. Archive hashes and CRC integrity are checked. Prediction also succeeds from a freshly extracted ZIP without the DUTS download or training cache.

The required interface is:

```
python predict.py --image photo.jpg --out mask.png
```

It saves a 256×256 PNG containing zero and 255, then prints:

```
T CPU ms: <measured average>
S disk bytes: 499643
```

The CLI averages 20 additional predictions after its initial prediction. This repeat is separate from the full benchmark average.

10.3 Scope and conclusion

The selected full-width network achieves **CES 0.6024** at **10.821 ms/image**, comfortably below the CPU limit. Its compact parameter representation improves cost efficiency while retaining essentially unchanged benchmark quality.

The captured masks remain approximate and the preserved source files are received WhatsApp copies rather than unmodified camera originals. Timing depends on the execution environment; no cross-hardware or statistical significance claim is made.

The solution uses Python's standard library, PyTorch, NumPy, OpenCV, and Matplotlib. No external pretrained segmentation model, library segmentation/morphology implementation, or separate inference engine is used.

References

[1] L. Wang et al., "Learning to Detect Salient Objects with Image-Level Supervision," *CVPR*, 2017. https://saliencydetection.net/duts/.

[2] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," *CVPR*, 2015. https://arxiv.org/abs/1411.4038.

[3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 2015. https://arxiv.org/abs/1505.04597.

[4] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. https://arxiv.org/abs/1704.04861.